REFERENZBLATT

Wie Sprachmodelle funktionieren

Die Definitionen, Zusammenhänge und Entscheidungskriterien des Workshops zum Nachschlagen im Arbeitsalltag. Die Ziffern am rechten Rand verweisen auf den Abschnitt des englischen Kurses *Tokens to Decisions*, der den Eintrag ausführlich behandelt.

Kurs (englisch) `../../course/index.html` **Foliensatz** `../../presentation/de/index.html`
Englische Fassung `../handout.html`

Teil Eins · Wie ein Sprachmodell funktioniert

> ZUR SPRACHE
> Der Kurs ist auf Englisch verfasst. Dieses Blatt verwendet die im deutschsprachigen Fachgebrauch üblichen Begriffe und nennt bei der ersten Erwähnung den englischen Originalbegriff. Der Abschnitt *Terminologie* am Ende enthält die vollständige Zuordnung beider Formen.

LERNEN UND TRAINING

Maschinelles Lernen 1.1 — die Regeln werden durch ein Suchverfahren ermittelt und nicht von einer Person geschrieben. Beim **überwachten Lernen** werden Eingaben zusammen mit den gewünschten Ausgaben vorgegeben. Beim **selbstüberwachten Lernen** werden die Zielwerte aus den Daten selbst gebildet; so werden Sprachmodelle trainiert: Der folgende Textabschnitt wird verdeckt und vorhergesagt. Eine Annotation entfällt, die Menge verfügbarer Trainingsdaten ist daher praktisch unbegrenzt.

Neuronales Netz 1.2 — Schichten einfacher Einheiten. Jede Einheit bildet eine gewichtete Summe ihrer Eingaben, addiert eine Konstante und wendet eine nichtlineare Funktion an. Ohne diese Nichtlinearität lassen sich gestapelte Schichten algebraisch zu einer einzigen Schicht zusammenfassen, und eine einzelne Schicht kann bereits das Exklusiv-Oder nicht darstellen.

Parameter 1.2 — die gelernten Zahlen: die **Gewichte**, mit denen jede Eingabe multipliziert wird, und der **Bias**, der zur Summe addiert wird. Ein aktuelles großes Modell hat in der Größenordnung von 10^{11} Parameter. Davon zu unterscheiden sind **Hyperparameter** — Anzahl der Schichten, Lernrate, Batch-Größe —, die vor dem Training festgelegt und nicht gelernt werden.

> $z = w_1 x_1 + w_2 x_2 + b$
> $\hat{y} = \sigma(z) = 1 / (1 + e^{-z})$
> Eine Einheit mit zwei Eingaben. Ein neuronales Netz besteht aus sehr vielen solcher Einheiten, in Schichten angeordnet.

Verlustfunktion 1.3, 1.9 — eine einzelne Zahl, die angibt, wie falsch eine Vorhersage war, und zugleich die vollständige Spezifikation des Trainingsziels: Das Modell lernt genau das, was diese Zahl verringert. Für einfache Regression eignet sich der quadratische Fehler $\frac{1}{2}(\hat{y} - y)^2$. Sprachmodelle verwenden die **Kreuzentropie**, also −log der Wahrscheinlichkeit, die dem tatsächlich aufgetretenen Token zugewiesen wurde. Sie bestraft eine Wahrscheinlichkeit nahe null für ein Ereignis, das dann eintritt.

Gradientenabstieg 1.4 — die Steigung der Verlustfunktion wird bezüglich jedes Parameters berechnet, anschließend wird jeder Parameter um einen kleinen Betrag in Richtung des Abstiegs verschoben. Ist die Schrittweite zu klein, verläuft das Training sehr langsam; ist sie zu groß, überschreitet jeder Schritt das Ziel. In der Praxis wird der Gradient aus einem kleinen zufälligen Batch geschätzt und nicht aus dem gesamten Datenbestand.

> $w \leftarrow w - \eta \cdot \partial L / \partial w$
> η ist die Lernrate. Das Training eines großen Sprachmodells wendet diese Regel wiederholt auf rund 10^{11} Parameter an.

Backpropagation 1.6 — ein Verfahren, das alle benötigten Ableitungen in einem einzigen Rückwärtslauf berechnet, mit etwa dem Aufwand eines zusätzlichen Vorwärtslaufs. Backpropagation berechnet Ableitungen; der Gradientenabstieg führt die Anpassung durch. Beide Begriffe werden häufig gleichgesetzt und bezeichnen verschiedene Vorgänge.

Generalisierung und Überanpassung 1.7 — maßgeblich ist das Verhalten bei Eingaben, die nie gesehen wurden. Ein Modell mit ausreichender Kapazität kann seine Trainingsdaten auswendig lernen und bei neuen Eingaben schlecht abschneiden; ein zurückgehaltener Datensatz macht dies sichtbar. Große Modelle behalten zudem wörtliche Ausschnitte der Trainingsdaten, was eine lizenzrechtliche Frage unabhängig von der Qualität ist.

Inferenz 1.8 — die Anwendung des trainierten Modells. Die Parameter sind unveränderlich, es wird nichts zurückgeschrieben, und keine Nutzung verändert das Modell. Was wie Erinnerung wirkt, ist stets Text, den die umgebende Software erneut übermittelt hat.

VON TEXT ZU VORHERSAGE

Token 2.1 — ein Textabschnitt aus einem festen Vokabular von etwa 100.000 bis 200.000 Einträgen. Das Vokabular wird vor dem Training gebildet, indem wiederholt das häufigste benachbarte Zeichenpaar zusammengefasst wird (**Byte-Pair-Encoding**). Die Häufigkeit bestimmt die Granularität: Häufige Wörter bilden ein Token, seltene mehrere. Ein führendes Leerzeichen gehört zum jeweiligen Token, und das Modell kann die einzelnen Zeichen innerhalb eines Tokens nicht erkennen — daher sind Aufgaben auf Zeichenebene unerwartet schwierig.

Spracheffekte 2.5 — das Vokabular wurde aus einem Korpus gebildet, in dem Englisch und Quellcode überwiegen. Derselbe Inhalt benötigt in den meisten anderen Sprachen mehr Tokens. Für Deutsch ist dieser Effekt relevant: Komposita und längere Wortformen werden in mehrere Tokens zerlegt, sodass ein deutscher Text in der Regel mehr Tokens benötigt als die englische Entsprechung. Das wirkt unmittelbar auf die Kosten und über das Kontextfenster darauf, wie viel Material hineinpasst. Der Faktor ist für die eigenen Texte zu messen und nicht zu schätzen.

Embedding 2.3 — die erste Schicht ist eine Tabelle mit einer Zeile je Vokabulareintrag. Die über die Token-Kennung ausgewählte Zeile ist das **statische Embedding**, geschrieben $\mathbf{e}(t)$. Das Training ordnet die Zeilen so an, dass ähnlich verwendete Tokens benachbart liegen.

> HÄUFIGE FEHLDEUTUNG
> Einzelne Dimensionen eines Embeddings entsprechen **keinen** interpretierbaren Eigenschaften. Bedeutungstragende Richtungen setzen sich aus vielen Koordinaten zusammen, und die üblichen zweidimensionalen Darstellungen sind Projektionen, die nahezu die gesamte Struktur verwerfen.

Kontextuelle Repräsentation 3.2 — geschrieben $\mathbf{h}_i^{(l)}$, der Vektor an Position *i* nach Schicht *l*, nachdem Attention Informationen aus anderen Positionen eingemischt hat. Ein statisches Embedding je Token, viele kontextuelle Repräsentationen: Das statische Embedding hängt nur vom Token ab, die kontextuelle Repräsentation vom Token *und seinem Satz*.

Attention 3.2, 3.3 — jede Position erzeugt eine Anfrage (query), die beschreibt, welche Information sie benötigt; jede Position bietet einen Schlüssel (key) an, der beschreibt, welche Information sie trägt; jede Position erhält eine gewichtete Mischung der Werte (values) der übrigen Positionen. Die Skalierung mit $\sqrt{d_k}$ hält die Softmax-Funktion außerhalb ihres gesättigten Bereichs, in dem die Gradienten verschwinden. Der Aufwand wächst quadratisch mit der Sequenzlänge; lange Kontexte sind deshalb rechenaufwendig.

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^\top / \sqrt{d_k} \right) V$$

Attention-Gewichte zeigen, woher Information bezogen wurde. Sie sind keine Erklärung der Ausgabe und sollten nicht als solche dargestellt werden.

Transformer 3.4 — gestapelte Blöcke, die jeweils Multi-Head-Attention mit einem Feed-Forward-Netz verbinden, mit Residualverbindungen und Normalisierung um beide. Attention bewegt Information zwischen Positionen; das Feed-Forward-Netz verarbeitet Information an einer Position. Residualverbindungen machen große Tiefe trainierbar.

Logits und Softmax 3.5 — der Vorwärtslauf endet mit einem Rohwert je Vokabulareintrag. Die Softmax-Funktion exponenziert und normiert diese Werte zu einer Wahrscheinlichkeitsverteilung mit der Summe eins.

Temperatur 4.3, 4.4 — teilt die Logits vor der Softmax-Funktion. Werte unter 1 verschärfen die Verteilung, Werte über 1 flachen sie ab. Das Abflachen hebt die Wahrscheinlichkeit des *gesamten* Vokabulars an, und dieser Rest ist groß: Bei etwa 200.000 Tokens summiert sich ein geringer Zuwachs je Token zum überwiegenden Teil der Wahrscheinlichkeitsmasse. **Top-p** beschränkt vor dem Ziehen auf die kleinste Menge von Tokens, deren Wahrscheinlichkeiten sich zu p summieren; deshalb wird das Verfahren in der Praxis zusammen mit der Temperatur eingesetzt.

$$p(t) = e^{z_t/T} \,/\, \Sigma\, e^{z_{t'}/T}$$

Eine höhere Temperatur fügt kein Wissen hinzu. Sie verteilt die Auswahl auf mehr Tokens, auch auf unpassende.

Autoregression 4.6 — ein Token je Vorwärtslauf. Das Token wird an den Kontext angehängt, anschließend läuft der gesamte Vorgang erneut. Daraus folgen drei Konsequenzen: Die Kosten wachsen mit der Kontextlänge, frühe Fehler verstärken sich, weil sie zur Eingabe werden, und erzeugter Text ist von zugeliefertem Text nicht mehr zu unterscheiden.

Kontextfenster 4.7 — die maximale Eingabe in Tokens. Der Platz ist begrenzt und wird geteilt: Anweisungen konkurrieren mit Material um denselben Raum. Eine veröffentlichte Länge beschreibt, was *entgegengenommen* wird, nicht, wie zuverlässig Material an beliebiger Stelle *verwendet* wird. Das Auffinden einer bestimmten Angabe wird mit wachsender Eingabe unzuverlässiger; der mittlere Bereich eines langen Kontexts ist der schwächste.

Halluzination 4.8 — flüssige, inhaltlich falsche Ausgabe. Kein Schritt des Verfahrens prüft eine Aussage gegen die Wirklichkeit; das Modell erzeugt eine plausible Fortsetzung, und plausibel ist nicht dasselbe wie zutreffend. Es handelt sich um denselben Mechanismus, der korrekte Antworten erzeugt, angewendet auf einen Bereich, in dem die Trainingsdaten ihn nicht eingeschränkt haben. Gegenmaßnahmen sind daher struktureller Art — Retrieval, Quellenangaben, Prüfung — und keine nachträgliche Korrektur.

Pretraining und Post-Training 5.1 — das Pretraining auf der Vorhersage des nächsten Tokens liefert das Wissen. Das **überwachte Nachtrainieren** vermittelt anschließend das Frage-Antwort-Format, und das **präferenzbasierte Nachtrainieren** (RLHF und verwandte Verfahren) prägt Tonfall, Hilfsbereitschaft und Ablehnungsverhalten — und damit auch eine messbare Neigung, der fragenden Person zuzustimmen. Sicherheitsverhalten ist eine antrainierte Disposition und keine technisch erzwungene Beschränkung.

SYSTEME UM DAS MODELL

Harness 5.4 — das Programm, das das Modell umgibt: Es stellt den Kontext zusammen, liefert die Tool-Definitionen, wertet die Ausgabe aus, führt Tool-Aufrufe durch und entscheidet über den Abbruch. Dasselbe Modell verhält sich in verschiedenen Harnesses unterschiedlich; vieles, was dem Modell zugeschrieben wird, legt der Harness fest. GitHub Copilot in VS Code ist ein solcher Harness.

Tool und Tool-Aufruf 5.8 — das Modell gibt eine strukturierte Anforderung aus, die ein Tool und dessen Argumente benennt; der Harness prüft die Berechtigungen, führt den Code aus und hängt das Ergebnis an den Kontext an. Das Modell führt nichts selbst aus. Tool-Beschreibungen sind Teil des Prompts, und Tool-Ergebnisse sind nicht vertrauenswürdige Eingaben.

Agentische Schleife 5.9 — beobachten, entscheiden, handeln, wiederholen, bis eine Abbruchbedingung erfüllt ist. Die Zuverlässigkeit hängt stark von der Rückmeldung innerhalb der Schleife ab: Kann die Schleife ihr Ergebnis prüfen, etwa durch Tests oder Nachrechnen, nähert sie sich dem Ziel; ohne diese Rückmeldung setzen sich Fehler leicht fort.

Retrieval (RAG) 5.11 — Dokumente werden einmalig eingebettet, zu einer Frage werden die nächstgelegenen Abschnitte ermittelt und in den Kontext eingefügt. Dies ist die wirksamste Einzelmaßnahme gegen Halluzination, weil sie Abruf aus dem Gedächtnis durch Lesen ersetzt. Die auftretenden Fehler sind Suchfehler; zur Fehlersuche ist daher zu prüfen, was abgerufen wurde, und nicht die Frage umzuformulieren.

Prompt Injection 6.1, 6.2 — Text, der dort platziert wird, wo ein Modell ihn liest, und als Anweisung formuliert ist. Bei der **indirekten** Variante steht der Text in einem Dokument, einer Webseite oder einem Datensatz aus externer Quelle. Das Verfahren funktioniert, weil Anweisungen und Daten denselben Eingabekanal verwenden und die Architektur die Herkunft nicht festhält. Es handelt sich um eine Eigenschaft der Architektur; eine allgemeine Korrektur gibt es derzeit nicht.

Was aus der Funktionsweise folgt — Zusammenfassung von Teil Eins

MECHANISMUS	FOLGE FÜR DIE PRAXIS
Parameter werden aus Beispielen gelernt	Ob ein Modell eine Aufgabe beherrscht, zeigt nur ein Test mit bekannten Antworten
Trainingsziel: das nächste Token vorhersagen	Ausgaben sind plausibel, aber nicht geprüft
Text wird in Tokens zerlegt	Buchstaben zählen und exaktes Rechnen sind unzuverlässig; Kosten und Kontext werden in Tokens gemessen; Deutsch braucht mehr Tokens
Das nächste Token wird gezogen	Dieselbe Anfrage kann verschiedene Antworten ergeben
Jedes erzeugte Token wird angehängt	Lange Kontexte kosten mehr; frühe Fehler setzen sich fort; die Herkunft von Text ist nicht gekennzeichnet
Feste Parameter, begrenzter Kontext	Kein Gedächtnis zwischen Dialogen; neuere und interne Informationen sind nur bekannt, wenn sie im Kontext stehen
Kein Schritt prüft Aussagen	Halluzinationen sind zu erwarten; Ergebnisse müssen geprüft werden
Ein Harness umgibt das Modell	Vieles, was man „der KI“ zuschreibt, ist eine Einstellung des Systems

Teil Zwei · Was daraus für die Praxis folgt

DIE BEGRIFFE IN GITHUB COPILOT

Wo die Begriffe in GitHub Copilot in VS Code vorkommen — Stand Oktober 2026, aus Sekundärquellen; Bezeichnungen ändern sich zwischen Versionen

BEGRIFF	IN GITHUB COPILOT IN VS CODE
Modell	Auswahlliste im Chat. Welche Modelle erscheinen, legt die Organisation fest.
Kontext	Angehängte Dateien und Ordner (mit `#` im Eingabefeld), der bisherige Chat, je nach Einstellung die geöffnete Datei.
Dauerhafte Anweisungen	Anweisungsdateien im Projekt, z. B. `.github/copilot-instructions.md`; Prompt-Dateien (`*.prompt.md`) für wiederkehrende Anfragen.
Arbeitsmodi	*Ask*: Fragen und Antworten. *Plan*: Plan vor der Ausführung. *Agent*: arbeitet selbstständig mit Tools.
Tools, agentische Schleife	Im Agent-Modus: Dateien lesen und ändern, Terminalbefehle ausführen. Erweiterbar über MCP-Server.
Technische Beschränkung	Freigabe von Tool-Aufrufen und Terminalbefehlen; Vorgaben der Organisation.
Kosten	GitHub AI Credits seit 1. Juni 2026, berechnet nach verbrauchten Tokens; 1 Credit = 0,01 US$. Die Code-Vervollständigung verbraucht keine Credits.

MODELLAUSWAHL

Welche Eigenschaft für welche Arbeit zählt — für die jeweilige Aufgabe zu gewichten, statt nach dem besten Modell zu suchen

EIGENSCHAFT	BESONDERS WICHTIG BEI
Leistung bei schwierigen Aufgaben	Mehrstufiger Arbeit, bei der sich ein früher Fehler durch alles Folgende fortsetzt
Befolgen von Anweisungen	Vorgaben, die über eine lange Ausgabe hinweg gelten müssen, nicht nur im ersten Absatz
Strukturierte Ausgabe	Einer vorgegebenen Form — Tabelle, Schema, feste Felder — zuverlässig bei jedem Durchlauf
Tool-Nutzung	Auswahl des richtigen Tools, sinnvollen Argumenten und korrekter Auswertung des Ergebnisses
Verarbeitung langer Kontexte	Material, das weit vom Ende des Fensters entfernt steht und dennoch verwendet werden muss
Antwortzeit	Dialogbetrieb mit häufigem Wechsel, bei dem Wartezeit den Arbeitsfluss unterbricht
Kosten	Allem, was wiederholt ausgeführt wird und wo sich ein kleiner Unterschied je Aufruf summiert

Reasoning Effort 8.1 — eine Einstellung je Aufgabe bei Modellen, die sie unterstützen, und keine feste Eigenschaft des Modells. Zusätzliche Berechnung zur Laufzeit hilft bei Aufgaben mit tatsächlicher Schlussfolgerungstiefe; bei Abruf oder Umformatierung erhöht sie Kosten und Antwortzeit, ohne die Antwort zu verbessern. Das Ausmaß des Nutzens hängt von Modell, Aufgabe und Budget ab und ist durch Vergleich an der eigenen Arbeit zu bestimmen, nicht durch Annahme.

Benchmarks 8.2 — als erster Filter geeignet, nicht als Entscheidungsgrundlage. Zusammengesetzte Indizes enthalten eine Gewichtung, die jemand anderes festgelegt hat; die Testbedingungen unterscheiden sich zwischen veröffentlichten Werten; Benchmark-Inhalte gelangen in spätere Trainingsdaten; gesättigte Benchmarks verringern sichtbare Unterschiede bis zur Zufallsschwelle; und keiner von ihnen bildet die eigene Aufgabe ab. Geeignet sind sie, um aus dreißig Kandidaten drei auszuwählen.

Eigene Evaluation 8.3, 14.2 — fünf bis zehn reale Aufgaben mit bekanntem Sollergebnis; Abnahmekriterien, die *vor* den Durchläufen festgelegt werden; je Aufgabe drei Durchläufe, da die Ausgabe gezogen wird und ein einzelner Durchlauf den Zufall misst; Bewertung anhand der Kriterien unter Erfassung von Kosten und Dauer. Der Aufwand beträgt etwa einen halben Tag und beantwortet die Frage, die eine Rangliste nicht beantworten kann.

KOSTEN

Fünf Einheiten, die regelmäßig verwechselt werden — die Umrechnung zwischen ihnen ist nicht festgelegt

EINHEIT	WAS SIE BEZEICHNET	FESTGELEGT DURCH
Token	Ein verarbeiteter Textabschnitt	Den Tokenizer — technisch, nicht kommerziell
Modellaufruf	Eine Anfrage mit eigenem Kontext	Die Architektur des Systems
Ihre Anweisung	Eine angeforderte Leistung	Sie — und sie kann viele Aufrufe auslösen
Credit	Eine Verrechnungseinheit mit festgelegtem Wert	Den Anbieter, durch Richtlinie und änderbar
Kontingent	Credits je Lizenz und Abrechnungszeitraum	Den Vertrag Ihrer Organisation

Eine Anweisung kann viele Modellaufrufe auslösen; ein agentischer Ablauf erzeugt aus einem einzelnen Satz leicht zwanzig. Wie Tokens in Credits umgerechnet werden, legt der Anbieter fest. 8.5

Token-Preise 8.4 — Eingabe, zwischengespeicherte Eingabe und Ausgabe werden getrennt berechnet. Die Ausgabe kostet in der Regel ein Mehrfaches der Eingabe, da sie sequenziell erzeugt wird, während die Eingabe parallel gelesen wird. Zwischengespeicherte Eingabe — ein Präfix, das der Anbieter bereits verarbeitet hat — kostet häufig etwa ein Zehntel der Eingabe. Daraus folgt der finanzielle Grund, dauerhafte Anweisungen stabil zu halten und Veränderliches ans Ende zu stellen. Tokens für Zwischenberechnungen werden als Ausgabe abgerechnet und sind in der Oberfläche häufig nicht sichtbar.

Was den Verbrauch erhöht. Lange angehängte Dokumente, die bei jedem Dialogschritt erneut übermittelt werden. Lange Dialoge, da der gesamte Verlauf je Schritt erneut übermittelt wird. Hoher Reasoning Effort. Agentische Schleifen, die sowohl Schritte als auch bei jedem Schritt Kontext hinzufügen. Eine Änderung am Anfang des Prompts, wodurch die Zwischenspeicherung entfällt. Und Wiederholungen aufgrund einer Aufgabe, die beim ersten Mal unzureichend spezifiziert war.

> **DER MASSGEBLICHE KOSTENANTEIL**
> Die Modellkosten sind in der Regel der kleinere Posten. Zwanzig Minuten Prüfung übersteigen sie, bewertet mit einem Vollkostensatz, meist um ein Mehrfaches. Der günstigste Weg ist daher derjenige, der die wenigsten Korrekturen erfordert. Zu berücksichtigen sind Modellkosten, Durchlaufzeit, Wiederholungen, menschliche Prüfung und die Kosten eines Fehlers, der unbemerkt bleibt. 8.7

CONTEXT ENGINEERING

Inhalt des Kontexts 9.1 — der System-Prompt des Anbieters, die Tool-Definitionen, Ihre dauerhaften Anweisungen, Material aus Ihrer Arbeitsumgebung, abgerufene Abschnitte, der bisherige Dialog und zuletzt Ihre Eingabe. Ihre Eingabe ist davon ein kleiner Anteil. Viele Qualitätsprobleme gehen auf den Kontext zurück, oft auf Bestandteile, die in der Oberfläche nicht sichtbar sind.

Lange Sitzungen. Harnesses behandeln das Anwachsen unterschiedlich: Manche verwerfen die frühesten Schritte, manche fassen fortlaufend zusammen, manche rufen gezielt ab, viele fügen dauerhafte Anweisungen bei jeder Anfrage erneut ein. Welches Verfahren das eigene System verwendet, bestimmt, was unbemerkt entfällt. Bei Arbeiten über mehrere Sitzungen ist jede Sitzung mit einer kurzen Übergabenotiz zu beenden und die folgende damit zu beginnen. 9.4

Sechs Bestandteile eines belastbaren Prompts — die beiden letzten fehlen am häufigsten und verändern das Ergebnis am stärksten

BESTANDTEIL	SCHWACH	BELASTBAR
Ziel	„Vergleiche die Lieferanten“	„Eine vierseitige Empfehlung für einen Lieferanten, für die Entscheidungssitzung am Donnerstag“
Hintergrund	*fehlt*	„Die Anlagen laufen im Dauerbetrieb; der Dauerdurchsatz ist wichtiger als der Spitzenwert“
Vorgaben	„Sei genau“	„Eine Herstellerangabe nie als Messwert darstellen“
Ausgabeform	„Schreib das auf“	„Vergleichstabelle, dann Widersprüche, dann Annahmen, dann Empfehlung“
Abnahmekriterien	*fehlt*	„Jede Zahlenangabe auf eine Seitenzahl oder einen Zeilenbereich zurückführbar“
Umgang mit Lücken	*fehlt*	„Aufführen, was nicht überprüft werden konnte und was stattdessen angenommen wurde“

WANN EIN NEUER DIALOG ZU BEGINNEN IST

- Die Aufgabe hat sich geändert.
- Drei Korrekturen sind erfolglos geblieben — der Kontext enthält nun drei falsche Ansätze.
- Die Qualität hat über eine lange Sitzung nachgelassen.
- Der Wechsel von der Erkundung zur eigentlichen Arbeit steht an.
- Vor Schritten, bei denen eine übrig gebliebene frühere Anweisung teuer wäre.

ARBEITSMODI UND AUTONOMIE

Vier Modi — zu beginnen ist mit dem geringsten Autonomiegrad, der die Aufgabe erfüllen kann

MODUS	SIE BEURTEILEN	GEEIGNET FÜR
Dialog	Jede Antwort	Verstehen, Entwerfen, Festlegen des Vorgehens
Planung	Den Plan, vor jeder Ausführung	Umfangreiche, unklare oder teuer zu wiederholende Aufgaben
Interaktiv	Jede Aktion, sobald sie vorgeschlagen wird	Die meisten umfangreicheren Arbeiten und jeden ersten Durchlauf
Autonom	Nur das Endergebnis	Eingeführte, prüfbare, wiederkehrende Aufgaben

Die Auswahl richtet sich nach sechs Fragen: Wie eindeutig ist die Aufgabe? Wie umkehrbar sind die Aktionen? Welche Kosten verursacht ein Fehler? Lässt sich der Erfolg automatisch prüfen? Worauf können die Tools zugreifen? Wie viel nicht dokumentiertes Urteilsvermögen ist erforderlich? In GitHub Copilot entspricht *Ask* dem Dialog und *Plan* der Planung; *Agent* arbeitet interaktiv oder autonom, je nachdem, ob Aktionen einzeln oder automatisch freigegeben werden. 10.2, 10.3

Drei Arten von Kontrolle, die nicht austauschbar sind — §10.5

ART	BESTIMMT	BEISPIEL
Anweisung Prompts, Vorgaben, Stilregeln	Was *wahrscheinlich* geschieht	„Zu jeder Zahl eine Seitenangabe nennen“
Orchestrierung Schritte, Verzweigungen, Abbruchbedingungen	Was *üblicherweise* geschieht	Ein Prüfschritt, der jede Quellenangabe kontrolliert, bevor der Entwurf zurückgegeben wird
Technische Beschränkung Berechtigungen, Sandbox, Freigabe	Was *möglich* ist	Kein Schreibzugriff außerhalb des Entwurfsordners

Die Formulierung „verändere keine Dateien“ ist eine Anweisung, und eine Anweisung kann durch alles andere im Kontext verdrängt werden, auch durch Text, der in einem Dokument eingetroffen ist. Ein Nur-Lese-Zugriff ist eine technische Beschränkung und kann es nicht. Alles, was teuer oder nicht umkehrbar ist, gehört in die dritte Zeile. In GitHub Copilot ist eine Anweisungsdatei eine Anweisung, die Freigabe von Tool-Aufrufen eine technische Beschränkung.

PLANUNG, ZERLEGUNG UND DELEGATION

Wann Planung ihren Aufwand rechtfertigt. 11.1 Die Aufgabe ist umfangreich, unklar oder teuer zu wiederholen; mehrere Beteiligte müssen sich darüber verständigen, was „fertig“ bedeutet; die Arbeit wird an jemanden oder etwas anderes delegiert; oder ein erster Versuch ist bereits unerwartet ausgefallen. Ein Planungsdurchlauf kostet einen Bruchteil eines vollständigen Versuchs und macht ein Missverständnis in einem kurzen Zwischenergebnis sichtbar statt in einem fertigen Arbeitsergebnis.

Ein brauchbarer Plan 11.2 gibt das Ziel in den Worten des Modells wieder, benennt die fehlenden Informationen, legt das Arbeitsergebnis fest, listet Vorgaben und Abnahmekriterien auf, zerlegt die Arbeit, benennt die Abhängigkeiten, schlägt je Teilaufgabe ein Modell und einen Modus vor und beschreibt die Zusammenführung. Die wiedergegebene Zielformulierung zu lesen ist die günstigste verfügbare Fehlererkennung: Ist sie dort falsch, ist alles Folgende ebenfalls falsch.

Wann Delegation sinnvoll ist. 11.4 Die Aufgabe übersteigt, was ein Kontext zuverlässig aufnimmt; die Teilaufgaben unterscheiden sich deutlich im Schwierigkeitsgrad, sodass für einen Teil ein günstigeres Modell genügt; Teile können tatsächlich parallel laufen; oder die Arbeit wiederholt sich häufig genug, um eine einmalige Spezifikation zu rechtfertigen.

Was Delegation kostet. Kontext wird zwischen Teilaufgaben nicht übertragen — jede beginnt ohne das, was die vorherige gelernt hat. Die Spezifikation ist eigener Aufwand. Fehler setzen sich fort, ohne erneut geprüft zu werden. Die Zusammenführung ist ein Beurteilungsschritt, der regelmäßig als reine Routine behandelt wird. Und sequenzielle Abhängigkeiten erhöhen die Durchlaufzeit, was die Parallelisierung nicht ausgleicht. Ein einzelnes leistungsfähiges Modell mit sorgfältig zusammengestelltem Kontext liefert häufiger bessere Ergebnisse als eine schlecht orchestrierte Kette.

DAUERHAFTE KONFIGURATION

Wohin eine Anweisung gehört — jeweils in den engsten Geltungsbereich, der noch alle Fälle erfasst

GELTUNGSBEREICH	AUFWAND	GEEIGNET FÜR
Einmalige Anweisung	Einmalig	Was an dieser einen Anfrage besonders ist
Persönliche Voreinstellungen	Jede Anfrage, überall	Schreibstil, Sprache, Detailgrad
Projektanweisungen	Jede Anfrage im Projekt	Terminologie, Konventionen, Standardaufbau
Pfadbezogene Anweisungen	Nur bei passender Arbeit	Regeln für eine bestimmte Materialart
Gespeicherte Prompt-Vorlage	Beim Aufruf	Eine wiederkehrende Anfrage in gleicher Form
Skill	Beschreibung immer; Inhalt bei Bedarf	Ein Verfahren mit festgelegten Anforderungen
Organisationsrichtlinie	Je nach Plattform	Was unabhängig von persönlichen Vorlieben gelten muss

Rangfolge der Anweisungen. 12.1 Anbieter trainieren Modelle in der Regel darauf, Anweisungen aus dem Systemkanal Vorrang vor Benutzereingaben einzuräumen und Benutzereingaben Vorrang vor Inhalten aus Dokumenten. Diese Ordnung hält überwiegend. Sie ist antrainiertes Verhalten und keine erzwungene Regel, und für Konflikte *innerhalb* einer Ebene existiert keine dokumentierte Auflösung. Widersprüche sind daher zu beseitigen und nicht einer Rangfolge zu überlassen.

Ein Stilprofil wirkt, wenn es Entscheidungen benennt 12.2, 12.4 — Stimme, Tempus, Person, Schreibweise von Überschriften, Satzlänge, zu vermeidende Wörter — statt gutes Schreiben allgemein zu beschreiben. „Schreibe klar“ ist zu allgemein, um etwas zu bewirken; „einen Absatz nie mit *Darüber hinaus* beginnen“ ist eine prüfbare Vorgabe. Das Profil ist vierteljährlich zu prüfen und zu kürzen: Jede Zeile wird mit jeder Anfrage übermittelt, und Anweisungen, die nicht mehr zutreffen, verdrängen solche, die zutreffen.

SKILLS, AGENTEN UND TOOLS

Neun Begriffe, die regelmäßig verwechselt werden — §13.2

BEGRIFF	STELLT BEREIT	WIRD AUSGELÖST DURCH
Prompt	Text, einmalig	Sie, durch Eingabe
Gespeicherte Prompt-Datei	Text, wiederverwendbar	Sie, über den Namen
Dauerhafte Anweisungen	Text, immer vorhanden	Niemanden — stets aktiv
Skill	Text und optional Dateien, bei Bedarf	Das Modell, sofern die Auswahl automatisch erfolgt
Eigener Agent	Anweisungen, eine Tool-Auswahl, teils ein Modell	Sie, durch Auswahl
Subagent	Dasselbe, innerhalb eines größeren Ablaufs	Das orchestrierende System
Tool	Die Fähigkeit, außerhalb von Text zu handeln	Modell fordert an; Harness führt aus
MCP-Server	Ein Paket aus Tools und Ressourcen	Einmalige Konfiguration
Hook	Code an festgelegter Stelle, bei jedem Durchlauf	Niemanden — deterministisch

Skills und gestufte Offenlegung. 13.2 Ein kurzer Name und eine Beschreibung verbleiben dauerhaft im Kontext; der Inhalt wird geladen, sobald der Skill als einschlägig beurteilt wird; zugehörige Dateien werden nur bei Bedarf geöffnet. Wo ein Harness Skills automatisch auswählt, trägt die Beschreibung die Hauptlast — sie ist dem Modell bei der Entscheidung meist die einzige verfügbare Information. Sie ist deshalb so zu formulieren, dass sie angibt, *wann* der Skill zutrifft, und nicht, was er enthält. Manche Umgebungen verlangen stattdessen den Aufruf über den Namen, was an der Gestaltung des Inhalts nichts ändert.

EINEN MCP-SERVER HINZUZUFÜGEN HEISST, SOFTWARE ZU INSTALLIEREN

Es handelt sich um fremden Code, der mit den erteilten Zugangsdaten ausgeführt wird. Zu klären sind der Herausgeber, eine feste Version, die einmalige Durchsicht der Tool-Beschreibungen — sie gelangen in den Kontext und sind eine Angriffsfläche für Prompt Injection —, die Beschränkung der Zugangsdaten auf das Notwendige und ein Verzeichnis des Installierten. Nur die Tools freigeben, die für die Aufgabe nötig sind. 13.4

Umfang der zu erstellenden Struktur — die letzte Zeile wird am häufigsten übergangen

VORGEHEN	EINRICHTUNG	ANGEMESSEN, WENN
Einzelne Anfrage im Dialog	Keine	Einmalige Arbeit, oder das Ziel ist noch offen
Strukturierte Aufgabenspezifikation	Minuten	Es darauf ankommt und der erste Versuch fehlgeschlagen ist
Gespeicherte Prompt-Vorlage	Unter einer Stunde	Dieselbe Anfrage sich in gleicher Form wiederholt
Dauerhafte Anweisungen	Eine Stunde zzgl. Pflege	Eine Vorgabe für viele verschiedene Aufgaben gilt
Wiederverwendbarer Skill	Ein halber Tag zzgl. Pflege	Ein mehrstufiges Verfahren mit Anforderungen regelmäßig genutzt wird
Agent mit Tool-Zugriff	Einrichtung und Berechtigungsprüfung	Die Arbeit Aktionen erfordert, die Text allein nicht leisten kann
Mehrere abgestimmte Agenten	Erheblich	Teilaufgaben sich stark unterscheiden oder tatsächlich parallel laufen
Herkömmliche Software	Oft geringer als erwartet	**Das Verfahren exakt und die Eingaben strukturiert sind**

Häufigkeit, Variabilität der Eingaben, Fehlerkosten, Anzahl der Beteiligten und Stabilität der Aufgabe verschieben die Antwort. Ein Modell rechtfertigt seine Kosten, wo Eingaben unregelmäßig sind, Urteilsvermögen erforderlich ist oder die Aufgabe sich nicht vorab exakt spezifizieren lässt. Trifft nichts davon zu, ist ein Skript günstiger, schneller und wiederholbar. 13.6

PRÜFUNG

Abnahmekriterien vor der Arbeit festlegen. 14.1 Nachträglich formuliert beschreiben sie das, was entstanden ist. Im Prompt genannt verbessern sie sowohl die Ausgabe als auch die Prüfung. Maßstab für ein Kriterium ist, ob eine andere Person als die verfassende es anwenden und zum selben Urteil gelangen könnte.

PRÜFBARKEIT EINPLANEN

Ein Dokument, in dem jede Zahl ihre Quelle nennt, ist in Minuten geprüft. Dasselbe Dokument ohne Quellenangaben erfordert eine Stunde, da zunächst zu ermitteln ist, was überhaupt zu prüfen ist. Quellenangaben zu verlangen senkt daher den Prüfaufwand erheblich.

Sieben Dimensionen und jeweils ein anwendbares Kriterium — §14.1

DIMENSION	EIN PRÜFBARES KRITERIUM
Sachliche Richtigkeit	Jede genannte Zahl stimmt mit der dafür angegebenen Seite oder dem Zeilenbereich überein
Vollständigkeit	Alle vereinbarten Kriterien erscheinen für alle Gegenstände, Lücken sind als Lücken gekennzeichnet
Nachvollziehbarkeit	Keine Aussage ohne Quelle; „nicht angegeben“, wo eine Quelle dazu schweigt
Rechnerische Richtigkeit	Berechnete Werte lassen sich bei unabhängiger Nachrechnung reproduzieren
Strukturelle Gültigkeit	Jeder geforderte Abschnitt ist vorhanden und nicht leer
Klare Abgrenzung	Angaben aus einer Quelle werden nie als Messwerte dargestellt
Wiederholbarkeit	Drei Durchläufe liefern dieselben *Zahlen*, wenn auch nicht denselben Text

Angemessener Prüfumfang. 14.3 Erkundende Arbeit erfordert eine Durchsicht. Material, das in die eigene Meinungsbildung eingeht, erfordert Stichproben bei allem, worauf man sich stützen wird. Arbeit, die an Kolleginnen und Kollegen weitergegeben wird, erfordert die Rückführung jeder Sachaussage auf ihre Quelle. Alles, was in eine Entscheidung, einen Vertrag oder eine externe Mitteilung eingeht, erfordert eine vollständige Prüfung mit unabhängiger Nachrechnung der Zahlen.

AKTIONEN, DIE BESONDERE SORGFALT ERFORDERN

- Alles Nicht-Umkehrbare: Versenden, Einreichen, Bestellen, Löschen.
- Alles, was auf Material aus externer Quelle angewendet wird.
- Alles mit Schreibzugriff auf eine gemeinsam genutzte Ablage.
- Zahlen, die in eine Entscheidung, einen Vertrag oder eine externe Mitteilung eingehen.
- Abläufe, die vertrauliche Daten, nicht vertrauenswürdige Inhalte und einen ausgehenden Kanal verbinden — eines der drei Elemente entfernen.
- Erste Durchläufe, auch nach einem Wechsel von Modell oder Konfiguration.

Was nicht einzugeben ist. 14.4 Gültige Zugangsdaten, personenbezogene Daten identifizierbarer Personen, Material, das Dritten gegenüber vertraulich ist, sowie Unterlagen unter rechtlicher Aufbewahrungspflicht. Gelangt ein Zugangsdatum in einen Prompt, ist es zu wechseln: Ein Dialogverlauf ist eine protokollierte Datenablage. Übliches Arbeitsmaterial der eigenen Organisation ist in der Regel unbedenklich, und darauf entfällt der größte Teil dessen, was tatsächlich eingefügt werden soll.

Terminologie

Die deutschsprachige Fachliteratur verwendet für einen Teil dieser Begriffe die englischen Ausdrücke. Dieses Blatt folgt dem jeweils üblichen Gebrauch. Die Tabelle ordnet beide Formen einander zu, damit englischsprachige Quellen — einschließlich des Kurses selbst — anschlussfähig bleiben.

ENGLISCH	IN DIESEM DOKUMENT	ANMERKUNG
machine learning	maschinelles Lernen	
self-supervised learning	selbstüberwachtes Lernen	Zielwerte stammen aus dem Text selbst
neural network	neuronales Netz	
weights / bias	Gewichte / Bias	„Bias“ unübersetzt, da etabliert
loss	Verlustfunktion	
cross-entropy	Kreuzentropie	
gradient descent	Gradientenabstieg	
backpropagation	Backpropagation	unübersetzt, da etabliert
overfitting	Überanpassung	
inference	Inferenz	
token / vocabulary	Token / Vokabular	
embedding	Embedding	„Einbettung“ ist seltener
contextual representation	kontextuelle Repräsentation	
attention	Attention	„Aufmerksamkeitsmechanismus“ bei Ersterwähnung
context window	Kontextfenster	
sampling	Sampling (Ziehen)	Auswahl des nächsten Tokens
temperature	Temperatur	
hallucination	Halluzination	

ENGLISCH	IN DIESEM DOKUMENT	ANMERKUNG
harness	Harness (Rahmenprogramm)	kein etablierter deutscher Begriff
tool / tool call	Tool / Tool-Aufruf	
agentic loop	agentische Schleife	
retrieval	Retrieval	
prompt injection	Prompt Injection	unübersetzt, da Fachbegriff
reasoning effort	Reasoning Effort	Umfang der Zwischenberechnung
guidance	Anweisung	eine der drei Arten von Kontrolle
orchestration	Orchestrierung	
enforcement	technische Beschränkung	
acceptance criteria	Abnahmekriterien	
progressive disclosure	gestufte Offenlegung	
skill / agent / hook	Skill / Agent / Hook	unübersetzt, da Produktbegriffe

Weiterführende Quellen

Die kommentierte Quellensammlung des Kurses mit 46 Einträgen, sortiert nach Verwendungszweck, steht unter `../../course/resources.html`. Die fünf wichtigsten Einstiegspunkte sind englischsprachig:

- **3Blue1Brown**, neuronale Netze und Transformer — eine anschauliche visuelle Darstellung dieses Stoffs und ein geeigneter Einstieg.
- **Andrej Karpathy**, *Deep Dive into LLMs like ChatGPT* für den gesamten Ablauf und *Neural Networks: Zero to Hero* für den Aufbau von Grund auf.
- **Anthropic**, *Building effective agents* — Entwurfsmuster und ein offener Abschnitt darüber, wann sie ungeeignet sind.
- **Simon Willison**, Sammlung zu Prompt Injection — fortlaufende Dokumentation eines ungelösten Problems.
- **OWASP Top 10 for LLM Applications** — anerkannte Systematik der Schwachstellen, geeignet für die Abstimmung mit einer Sicherheitsabteilung.

Produktdokumentation. Für alles, was vom Tarif, der Version oder der Preisgestaltung abhängt, ist die jeweils aktuelle Dokumentation des Anbieters heranzuziehen und nicht dieses Blatt. GitHub Copilot, die Agentenkonfiguration von VS Code, das Model Context Protocol und die Agent-Skills-Spezifikation veröffentlichen jeweils eine eigene Referenz, die häufiger aktualisiert wird, als ein Kurs es kann.

> **HINWEIS ZUR ÜBERPRÜFUNG**
> Produktverhalten und alle kommerziellen Angaben dieses Blatts wurden im Oktober 2026 aus Sekundärquellen zusammengestellt, da die primären Dokumentationsserver aus der Arbeitsumgebung nicht erreichbar waren. Die Umstellung auf GitHub AI Credits zum 1. Juni 2026 beruht auf Suchergebnissen zur Ankündigung von GitHub vom April 2026. Vor einer verbindlichen Festlegung sind die Angaben zu bestätigen. Vier Fragen kann nur die eigene Administration beantworten: welches Abrechnungsmodell der Vertrag vorsieht; wie hoch das Kontingent ist und ob es organisationsweit zusammengefasst wird; ob eine Überschreitung zugelassen ist und was bei Erschöpfung geschieht; und welcher Bericht den Verbrauch wem gegenüber ausweist.